

0.54.3. La fonction `vitesse()` — régler la vitesse des déplacements

`#include <vitesse>` : `vitesse(ms)` règle le temps d'un pas pour `deplace_x`, `deplace_y`, `deplace` et `va_a` qui viennent après.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `vitesse(ms)` dit combien de **millisecondes** dure un pas des déplacements qui suivent. 250 au départ (4 pas par seconde) ; 100, plus vite ; 500, plus lentement. Elle ne fait rien bouger elle-même.

La ligne à écrire : `#include <vitesse>`.

Son argument : la durée d'un pas, écrite en clair.

Ce qu'elle coûte : une dizaine d'octets : elle range un nombre.

Essaie : `vitesse(500);` : le A traîne.

On retrouve vitesse dans 6 leçons

[0.55](#), [0.55.1](#), [0.55.2](#), [0.56](#), [0.56.1](#), [0.56.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1072 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	60 o	← la console, à l'allumage – la mise en route de la console
deplace_x()	158 o	← main() – une fonction fournie par le langage
main()	32 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles – la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 2	26 o sur place, dans deplace_x() + AdresseCase (10 o)
effacer()	× 1	15 o sur place, dans deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()	× 2	6 o sur place, dans deplace_x(), main() + AttendreImage (28 o)

```
Tes fonctions appelées :  
  deplace_x() x 1    19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <vitesse> : régler la vitesse des déplacements ----  
// Le même trajet que pour deplace_x, mais à 100 ms par pas : deux fois et demie plus vite.  
  
#include <vitesse>      // vitesse() : la durée d'un pas  
#include <deplace_x>  
#include <ALPHABET>
```

2. UNE VARIABLE — X

```
uint8_t x = 2;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

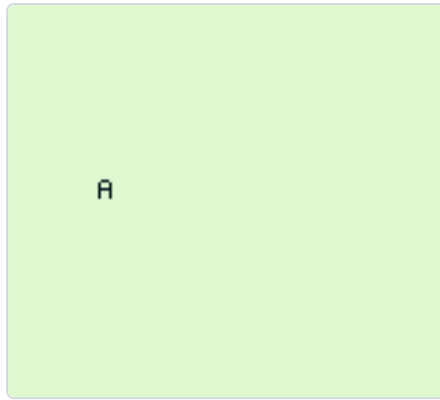
```
int main() {  
    vitesse(100);                // 100 ms par pas, au lieu de 250  
    deplace_x(x, 8, ALPHABET[0], 10); // les 10 pas vont plus vite  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



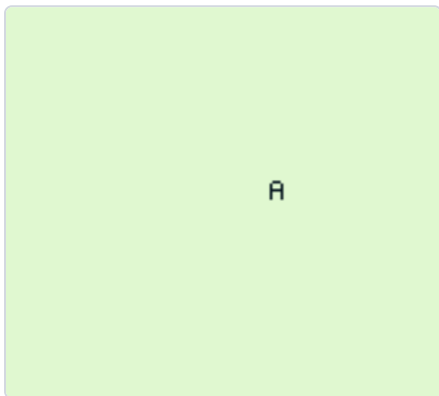
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le A traverse dix cases en une seconde.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ en 70 images (un peu plus d'une seconde), le A est arrivé en (12, 8) (12,8)

Ensuite : **0.55. La vitesse des déplacements : vitesse** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++